

LAPORAN PRAKTIKUM ALGORITMA PEMROGRAMAN
PERULANGAN FOR



Oleh :

MUHAMMAD ARIF

NIM 2511532017

MATA KULIAH ALGORITMA PEMROGRAMAN

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 27 OKTOBER 2025

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan praktikum **“Perulangan For ”** ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi hasil kegiatan praktikum yang bertujuan untuk memahami konsep perulangan **for** di java dalam pemrograman. Penulis menyadari bahwa pemahaman mendalam terhadap topik ini merupakan fondasi essential bagi pengembangan keterampilan pemrograman yang lebih advanced.

Penulis mengucapkan terima kasih sebesar-besarnya kepada:

1. Bapak DR. Wahyudi, S.T, M.T selaku pembimbing mata kuliah algoritma pemograman, yang telah memberikan bimbingan, ilmu, dan motivasi selama proses pembelajaran.
2. Asisten praktikum yang telah membimbing dan membantu penulis selama sesi praktikum berlangsung.

Padang, 27 Oktober 2025

Muhammad Arif

DAFTAR PUSTAKA

LAPORAN PRAKTIKUM ALGORITMA PEMROGRAMAN	i
KATA PENGANTAR.....	2
DAFTAR PUSTAKA.....	3
BAB I PENDAHULUAN	4
1. Latar Belakang	4
2. Tujuan.....	4
3. Manfaat	5
BAB II PEMBAHASAN	6
2.1 Landasan Teori	6
2.2 Langkah Pengerjaan.....	7
BAB III KESIMPULAN	15
3.1 Kesimpulan	15
3.2 Saran	15

BAB I

PENDAHULUAN

1. Latar Belakang

Sebagai mahasiswa pemula dalam pemrograman, saya menyadari bahwa teori algoritma sering terasa abstrak jika tidak langsung dipraktikkan. Konsep baru benar-benar “masuk” ketika saya mengetik kode dan melihat hasilnya secara nyata.

Salah satu fondasi penting dalam pemrograman adalah perulangan. Dalam kehidupan sehari-hari, banyak tugas yang bersifat berulang—seperti mencetak nomor urut, menghitung total nilai, atau menampilkan daftar item. Menulis kode yang sama berulang kali jelas tidak efisien. Di sinilah perulangan **for** dalam Java hadir sebagai solusi: memungkinkan kita menjalankan blok kode berulang kali dengan jumlah iterasi yang jelas dan terkendali.

Struktur **for** sangat cocok digunakan ketika jumlah perulangan sudah diketahui sejak awal. Praktikum ini bertujuan untuk memperkuat pemahaman tentang cara kerja **for** melalui eksperimen langsung, mengubah konsep teoretis menjadi keterampilan aplikatif, serta melatih pola pikir logis dalam menyelesaikan masalah yang bersifat repetitif.

2. Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Mengetahui dan mengaplikasikan penggunaan statement **for** di java.
2. Mampu menggunakan struktur **for** untuk mengeksekusi blok kode secara berulang dengan jumlah iterasi yang telah ditentukan.
3. Melatih kemampuan berpikir logis dalam menyelesaikan masalah yang bersifat repetitif tanpa menulis kode secara manual berulang kali.

3. Manfaat

Manfaat yang diperoleh dari praktikum ini adalah

1. Melatih logika pemrograman melalui pembuatan pola, pengulangan tugas, dan pengolahan data sederhana menggunakan jumlah iterasi tetap.
2. Meningkatkan keterampilan menulis kode yang efisien, karena praktikum menunjukkan betapa for menghindarkan penulisan kode yang redundan.

BAB II

PEMBAHASAN

2.1 Landasan Teori

Dalam pemrograman, seringkali diperlukan pengulangan eksekusi suatu blok kode untuk menyelesaikan tugas yang bersifat repetitif. Bahasa Java menyediakan tiga jenis struktur perulangan (*looping*), yaitu `for`, `while`, dan `do...while`. Di antara ketiganya, perulangan `for` merupakan struktur yang paling umum digunakan ketika jumlah iterasi telah diketahui sejak awal.

Menurut Duniaikom (2025), struktur perulangan `for` adalah instruksi yang dirancang untuk mengulang eksekusi satu atau beberapa baris kode dalam jumlah iterasi tertentu. Agar perulangan `for` dapat berjalan dengan benar, terdapat tiga komponen utama yang harus didefinisikan:

1. Inisialisasi (*start*)

Merupakan bagian awal perulangan yang biasanya berisi deklarasi dan pemberian nilai awal pada *variabel counter* (misalnya `i = 1`). Variabel ini digunakan untuk mengontrol jumlah pengulangan.

2. Kondisi (*condition*)

Merupakan ekspresi boolean yang dievaluasi sebelum setiap iterasi. Selama kondisi ini bernilai `true`, perulangan akan terus berlangsung. Jika kondisi bernilai `false`, perulangan dihentikan.

3. Increment/Decrement (*update*)

Merupakan bagian yang menentukan perubahan nilai variabel counter setelah setiap iterasi. Umumnya berupa operasi penambahan (`i++` atau `i = i + 1`) atau pengurangan (`i--` atau `i = i - 1`), namun juga dapat berupa penambahan dengan kelipatan tertentu (misalnya `i = i + 3`).

Dengan struktur umum sebagai berikut :

```
for (inisialisasi; kondisi; increment/decrement) {  
    // blok kode yang diulang  
}
```

2.2 Langkah Pengerjaan

a. Perulangan for 1

1. Buat package dan *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, buat struktur perulangan dengan format

```
for (inisialisasi; kondisi; increment/decrement) {  
    // blok kode yang diulang  
}
```

```
package Pekan5;  
  
public class PerulanganFor1 {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        for (int i = 1; i <=10; i++ )  
            System.out.println(i);  
    }  
}
```

Gambar 2.2. 1

3. Dengan inisialisasi $i=1$, kondisi $i \leq 10$ (jika kondisi tidak terpenuhi akan diulang terus), dan $i++$ (nilai i terus bertambah 1 setiap perulangan)
4. Dengan output sebagai berikut

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

Gambar 2.2. 2

b. Perulangan for 2

1. Buat *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, buat struktur perulangan dengan format

```
for (inisialisasi; kondisi; increment/decrement) {  
    // blok kode yang diulang
```

```

    }

    package Pekan5;

    public class PerulanganFor2 {

        public static void main(String[] args) {
            // TODO Auto-generated method stub
            for (int i = 1; i <=10; i++ )
                System.out.print(i + " ");

        }

    }

```

Gambar 2.2. 3

3. Dengan inisialisasi i=1, kondisi i<=10 (jika kondisi tidak terpenuhi akan diulang terus), dan i++ (nilai i terus bertambah 1 setiap perulangan)
4. Karena menggunakan print ,maka output akan diprint ke samping

```

1 2 3 4 5 6 7 8 9 10

```

Gambar 2.2. 4

c. Perulangan for 3

1. Buat *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, buat struktur perulangan dengan format

```

for (inisialisasi; kondisi; increment/decrement) {
    // blok kode yang diulang
}

```

```

package Pekan5;

public class PerulanganFor3 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int jumlah=0;
        for (int i = 1; i <=10; i++ ) {
            System.out.print(i);
            jumlah = jumlah + i;
            if (i<10) {
                System.out.print("+");
            }

        }System.out.println();
        System.out.println("jumlah = "+ jumlah);
    }
}

```

Gambar 2.2. 5

3. Dengan inisialisasi i=1, kondisi i<=10 (jika kondisi tidak terpenuhi akan diulang terus), dan i++ (nilai I terus bertambah 1 setiap perulangan)
4. Dengan menggunakan statement if, jika i<10 maka akan diprint “+”
5. Dengan output sebagai berikut

```

1+2+3+4+5+6+7+8+9+10
jumlah = 55

```

Gambar 2.2. 6

d. Perulangan for 4

1. Buat *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, ketikkan **import** java.util.Scanner. agar java mengetahui bahwa kita menggunakan *Scanner* pada pemrograman.
3. *Scanner* keyboard = **new** *Scanner* (System.in); digunakan untuk membaca input dari keyboard.
4. Setelah *class* dibuat, buat struktur perulangan dengan format

```

for (inisialisasi; kondisi; increment/decrement) {
    // blok kode yang diulang
}

```

```

package Pekan5;

import java.util.Scanner;

public class PerulanganFor4 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int jumlah = 0;
        int batas;
        Scanner input = new Scanner (System.in);
        System.out.print("masukkan nilai batas =");
        batas = input.nextInt();
        input.close();
        for (int i=1;i<=batas;i++){
            System.out.print(i);
            jumlah= jumlah+i;
            if (i<batas) {
                System.out.print("+");
            }else {
                System.out.print("=");
            }
        }
        System.out.println(jumlah);
    }
}

```

Gambar 2.2. 7

5. Pada program kali ini kita akan menentukan batasnya sendiri dengan memasukkan input berupa angka.
6. Jika $i < \text{batas}$ maka akan diprint “+” dan jika $i > \text{batas}$ maka akan diprint “=”
7. Dengan output sebagai berikut

```

masukkan nilai batas =10
1+2+3+4+5+6+7+8+9+10=55

```

Gambar 2.2. 8

e. Nested For 0

1. Buat *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, buat struktur perulangan dengan format


```

for (inisialisasi_luar; kondisi_luar; increment_luar) {
    for (inisialisasi_dalam; kondisi_dalam; increment_dalam) {
        // blok kode yang dijalankan dalam setiap iterasi dalam
    }
    // (opsional) kode setelah perulangan dalam selesai, tapi masih dalam iterasi
    luar
}
            
```

```

package Pekan5;

public class nestedFor0 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        for (int line = 1; line <= 5; line++) {
            for (int j = 1; j <= (-1 * line + 5); j++) {
                System.out.print(".");
            }
            System.out.print(line);
            System.out.println();
        }
    }
}

```

Gambar 2.2. 9

3. Dengan penjelasan sebagai berikut.
 - a. Jika $i=1$: $-1*1+5 = 4$
 - b. Jumlah titik yang akan dicetak adalah 4 buah
 - c. Jika $i=2$: $-1*2+5=3$
 - d. Jumlah titik yang akan dicetak adalah 4 buah
 - e. Jika $i=3$; $-1*3+5=2$
 - f. Jumlah titik yang akan dicetak adalah 4 buah
 - g. Jika $i=4$; $-1*4+5=1$
 - h. Jumlah titik yang akan dicetak adalah 4 buah
 - i. Jika $i=5$; $-1*5+5=0$
 - j. Jumlah titik yang akan dicetak adalah 4 buah
4. Setelah titik dicetak semua maka angka akan dicetak dibelakangnya
5. Dengan output sebagai berikut

```

....1
...2
..3
.4
5

```

Gambar 2.2. 10

f. Nested For 1

1. Buat *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, buat struktur perulangan dengan format
`for (inisialisasi_luar; kondisi_luar; increment_luar) {`

```

for (inisialisasi_dalam; kondisi_dalam; increment_dalam) {
    // blok kode yang dijalankan dalam setiap iterasi dalam
}
// kode setelah perulangan dalam selesai, tapi masih dalam iterasi luar
}

```

```

package Pekan5;

public class nestedFor1 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        for(int i =1; i<= 5; i++) {
            for(int j = 1; j<= 5; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}

```

Gambar 2.2. 11

3. Dengan penjelasan sebagai berikut
 - a. Program ini pakai dua perulangan for yang saling bersarang.
 - b. Perulangan luar (i) jalan 5 kali (artinya nanti akan ada 5 baris).
 - c. Setiap kali perulangan luar jalan, perulangan dalam (j) langsung jalan 5 kali juga —(jadi tiap baris isinya 5 bintang).
 - d. Karena pakai `System.out.print("*")`, bintang-bintang itu dicetak sebaris, nggak langsung turun ke bawah.
 - e. Setelah 5 bintang selesai dicetak, `System.out.println();` dipanggil biar kursor pindah ke baris baru.
4. Dengan output sebagai berikut

```

*****
*****
*****
*****
*****

```

Gambar 2.2. 12

g. Nested For 2

1. Buat *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, buat struktur perulangan dengan format

```

for (inisialisasi_luar; kondisi_luar; increment_luar) {
    for (inisialisasi_dalam; kondisi_dalam; increment_dalam) {
        // blok kode yang dijalankan dalam setiap iterasi dalam
    }
    // kode setelah perulangan dalam selesai, tapi masih dalam iterasi luar
}

```

```

package Pekan5;

public class nestedFor2 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        for(int i =0; i<= 5; i++) {
            for(int j = 0; j<= 5; j++) {
                System.out.print(i+j+ " ");
            }
            System.out.println();
        }
    }
}

```

Gambar 2.2. 13

3. Dengan penjelasan sebagai berikut

- a. Program ini menggunakan dua perulangan for yang bersarang: i (luar) dan j (dalam).
- b. Perulangan luar (i) jalan dari 0 sampai 5 total 6 kali (karena termasuk angka 0).
- c. Setiap kali i jalan, perulangan dalam (j) juga jalan dari 0 sampai 5 juga 6 kali.
- d. Di setiap iterasi dalam, program mencetak hasil dari i + j, lalu spasi.
- e. Karena i dan j mulai dari 0, angka yang muncul di baris pertama adalah: 0+0=0, 0+1=1, 0+2=2, ..., sampai 0+5=5.
- f. Di baris kedua (i = 1), hasilnya: 1+0=1, 1+1=2, 1+2=3, ..., sampai 1+5=6.
- g. Begitu seterusnya sampai i = 5, sehingga baris terakhir berisi: 5, 6, 7, 8, 9, 10.
- h. Setelah setiap baris selesai (setelah j selesai), System.out.println() dipanggil agar pindah ke baris baru.

- i. Hasil akhirnya adalah tabel angka berukuran 6×6 , di mana setiap sel berisi jumlah dari nomor baris (i) dan nomor kolom (j).
4. Dengan output sebagai berikut

0	1	2	3	4	5
1	2	3	4	5	6
2	3	4	5	6	7
3	4	5	6	7	8
4	5	6	7	8	9
5	6	7	8	9	10

Gambar 2.2. 14

BAB III

KESIMPULAN

3.1 Kesimpulan

Perulangan adalah salah satu konsep dasar yang sangat penting dalam pemrograman. Dalam menggunakan perulangan, penting untuk memperhatikan penggunaannya agar tidak mengakibatkan terjadinya infinite loop atau pengulangan yang tidak terbatas. Dengan menguasai konsep perulangan, kita dapat membuat program yang lebih efisien dan mudah.

3.2 Saran

Untuk meningkatkan pemahaman yang lebih mendalam tentang perulangan for, maka dari itu ada beberapa saran yang bisa dilakukan:

- Perbanyak variasi latihan perulangan for, terutama yang menggabungkan perhitungan matematis atau pola bintang, agar pemahaman tentang logika iterasi dan penggunaan variabel counter semakin kuat.
- Latih diri untuk menulis algoritma sederhana sebelum mengetik kode, misalnya dengan membuat sketsa alur atau tabel iterasi, agar lebih mudah memprediksi hasil dan menghindari kesalahan logika.

Masih banyak kekurangan dalam pembuatan laporan praktek ini, maka dari itu penulis sangat terbuka untuk menerima saran dari pembaca sekalian.